OS REF SHEET

GREP

Preemption = CPU given to another process (interrupt)
Non-preemption = cooperative (no interruptions)
Starvation = some ps. receive no CPU.

- -E = extended regex ; -r = extended regex (older)
- -e = pattern / expression (grep -e "abc" -e "def")
- -v = invert match
- -i = ignore case
- -n = line nr
- -c = count
- -r = recursive (subdir.) (grep -r "main")
- -x = whole line
- -o = matching part only
- -q = quiet mode

grep -q "^$i:" file → take usernames
grep -E '\<[A-Z]' file → words starting with cap.
grep -E '^[^b]*a[^b]*$' → contains a, not b (grep -E "a" -e "^[^b]")
grep "a" file | grep -v "b"
grep -c "^$" file → nr. of empty lines
grep -E -v -c "." → lines containing no characters
grep "[aA]" filename
grep -E "[aA]+" filename → words beginning with uppercase / lowercase vowels.
grep -Ei '(^|)[aeiouAEIOU][a-zA-Z]*($|)' file
grep -Ei '\<[aeiou][a-z]*\>' file
grep "." file → non-empty lines
grep -E '^(.*[aeiou]){3}.*$' → at least 3 vowels
grep -E '.*([01]*0){2}){2}.*' | grep -E '[01]{5,}$'
↳ multiple of binary 4?
grep -E "^(..)*.$" file → an odd nr. of chars.
grep -E -v "^(...)*$" → nr. not having nr. chars : 3 (grep -E -v "^(.{3})*$")

SED

S/regex/string/flag/
s/old/new
-r = extended regex (older)
E = extended regex ; -n = don't print automatically
-f = commands file
p = print ; !p ;
d = delete
a = append
i = insert
g = global → all occurences
sed -n "/^$/p" file | wc -l → nr. empty lines
sed -E 's/([^]*) ([^]*)/\1\2/g' file → delete first, third, fifth space...
sed -E "/7/d" filename → delete lines containing 7
sed -n "/7/p" filename → print lines not containing 7
sed -n '/^[^0-9]*$/p' file → no digits.
sed -E 's/[a-z]+//' → delete first ≠0 sequence (extended)
sed 's/[a-z]\+//' → delete first ≠0 sequence (default)

sed 's/a ls -l /.../' execvp(argv[1], argv+1)

SED + capturing groups
sed -E 's/([0-9])([0-9])/\2\1/g' → invert pairs of neighbouring digits
sed -Ei 's/^([a-z]+) ([a-z]+) (.*) ([a-z]+) $/\1\4\3\2/' file
sed -E "s/ +/\n/g" → replace all spaces with a newline

AWK

NR = current line nr.
NF = nr. of fields
$i = field i
$0 = whole line → {print $0}
BEGIN; END → BEGIN {...}{...} END {...}
{s=0; for(i=1; i<=NF; i++) s+=$i}
(sum of fields = line nr.)
if (NR<=NF) s+=$NR END {print s}
regex: $1 ~ /regex/ → check if text inside field 1 respects the regex
F: → field separator (:)
length(string) → returns len of str
split(string, array, sep)
index(s1, s2) → returns occurrence of s2 in s1 (first)
↳ 0 if not found ↳ integer, strstr(c) returns ptr
awk '{print $1 + $2}' file → sum of fields
awk 'BEGIN {c=0} /^$/ {c++} END {print c}' file → nr. empty lines
awk '{s+=$(NF-1)} END {print s}' file
awk '{print $2 * $(NF-1)}' file
sed ... a.txt > temp.txt && mv temp.txt a.txt
grep -Eq '\<cat\>' "$f" && echo "$f"

PAGE TABLE: uses page nr. to find frame
SEG. TABLE: uses seg. nr. to find base of that segment

LOGICAL MEMORY => PAGES (fixed-sized blocks)
PHYSICAL MEMORY => FRAMES (fixed-sized blocks)
PAGE TABLE: map page nr. to frame nr.

ensure mutual exclusion (function calls)
- pthread_mutex_lock(&m)
- sem_wait(&s)
- pthread_rwlock_wrlock(&rwl)

wc -c file	
ls -lh file	
du -h file	
stat -c %s file	

stat → metadata
du → disk usage
ls → show info about files
-l → long format (permissions, owner, date...)
-h → human readable

MEMORY ALLOCATION

- FIXED = RAM divided beforehand
- DYNAMIC = partitions created as needed

1) MBS FIXED PARTITION ALLOCATION: l.a = p.a.
→ OS performs no translation; contiguous alloc
⊕ faster (predefined RAM regions) (addr. known)
⊖ poor memory utilisation (fragmentation)
2) RELOCATABLE FIXED PARTITION ALLOCATION
p.a = base + l.a
→ contiguous alloc
3) DYNAMIC PARTITIONS → partitions created as needed ⊖ external fragm; p.a = base + v.a
=> FFit, WFit, BFit, NFit ; contiguous alloc
4) PAGING (non-contiguous alloc.) l.a = b + off.
⊕ no ext. fragm; fixed-size blocks; page table maps the virtual addr. with the phys. addr ⊖ slower; ⊖ memory page/process; PHYS. MEM = array of fixed-sized slots.
5) SEGMENTATION (non-contiguous alloc.)
⊖ ext. fragm; var-size blocks; segment table ⊕ fast ⊕ code share
6) SEGMENTATION + PAGING <segment, page, offset>; p.a = b + offset.
↳ each seg. div. into pages; seg. table + page table

seg. nested alloc.
⊕ flexibility in memory alloc. (can allocate mem. for ≠ sizes)
page alloc ⊕ fast
⊖ fixed-size restrictions

virtual addr <page, offset> → actual mem. addr (page table)

INT. FRAGM. = alloc blocks > than requested => waste, EXT. FRAGM. = ∃ no contig. mem in total > (e.g.

HARD / SYMBOLIC LINK

Hard link = dir. entry that references an inode
hard link → inode
inode ∈ one filesys.
≠ partitions => ≠ filesys
only the dir. entry removed if ∃ other links still.
Symbolic link = file containing target pathname
=> can span across ≠ filesystems.
- invalid if target file deleted; behaves like target file

ln a b
- => cannot span accross ≠ filesys.
- valid if ∃ at least one link to the inode
- deleted file (=) no more links to it (ref. count = 0)

ln -s a b

TEST

-d → directory ; -e → existence ; -f → file
-r → readable ; -L → symbolic path
-lt, -le, -gt, -ge, -eq, -ne,
-z string => true if string is empty (zero) → test -z "$str" | test "$str" == ""
-n string => true if string is NOT empty (non-zero) ↳ ! test -n "$str"
["$var1" = "$var2"] <=> test "$var1" = "$var2"
string1 \> string2 => true if string1 is lexicographically greater
string1 \< string2 => true if string1 is lexicographically smaller
-o → logical OR , -w → writable [[-r "$f" || -w "$f"]]

readable / writable [-r "$f" -o -w "$f"]

CACHES

① DIRECT cache → store the page in location:
page_addr % cache_size
⊕ fast, simple to search
⊖ collisions ⊖ no pages w same index
② ASSOCIATIVE cache → place each page on a free location formed through iterating the cache
⊖ slow
③ SET-ASSOCIATIVE caches → HT, cache organised in sets of pages, 1) take each cache; 2) compute page % nr. sets 3) group results in sets; 4) how many pages has each set 5) compare with max. nr. of pages 6) if one set overflows, evicts needed; no evicts = good performance

⊕ flexible ← dir
⊕ no collisions
⊖ does not use all cache lines

UNIX CLASSES

[:digit:]
[:alpha:]
[:alnum:]
[:lower:]
[:upper:]

Sort

-r = reverse
-n = numeric
-k = month
-u = unique

uniq

sort f | uniq
sort f | uniq -c
sort f | uniq -u

SHELL

argv ← $@ → all args as ARRAY
$* → all args as STRING
argc ← $# → nr. of arguments
$? → exit code of prev.
*.txt → all text files in the current dir.
-e → exists
-r → readable
-p → prompt
-rm → remove

( ) => array
$( ) => string

' ' → taken literally
" " → expand $
↳ whenever a var. might contain spaces or is empty
#! /bin/bash
↳ shebang + interpreter
"$var" → safer.
$((2+3))
`command` executes command and replaces it with the result of the execution
shift → args to the L
chmod +x file
↳ add execute permiss.
-x → remove execute p.
if ...; then ...; fi
$var → access value
for var in ...; do ... done
echo → print to terminal
k=$((k+1)) k=$((k-1))
"$var1" = "$var2"
${#str} → length

while true; do read -p "Dir: " d; if [! -d "$d"], then break; fi done

while read f1, f2; do if [-e "$f1"]; then run "$f2" done < a.txt

while read f1 f2 f3; do if [-f ...]; then cat "$f1" "$f2" > "$f3" fi done < a.txt

for f in *.txt; do grep -q "cat" "$f" && echo "$f" done
name of each .txt file in the current dir. containing "cat"

cat $var → cat "$var" → cat $var1 $var2 → cat file1 file2 > file3 → cat file1 file2 | cat file3 → cat file1 file2 | cat file3 > file4

$var vs "$var" = $var vs "$var" = $var vs "$var" = $

OPEN / CLOSE / POPEN / PCLOSE / FOPEN / FCLOSE / READ / WRITE

- open(name, flags, rights) → low-level, file descriptor → int fd = open("a.txt", O_RDONLY);
  - → open succeeds, read fails if file is directory
  - → open fails quickly if file ∄ or has no permissions.
  - → read is quick/cached if file is regular, readable
  - → read may block if file is FIFO/device/terminal
- flags: O_RDONLY, O_WRONLY,
- close(file descriptor)
- fopen(path, rights) → high-level, file descriptor → FILE *f = fopen("a.txt", "r")
- popen(cmd, rights) = pipe + fork + exec → FILE *p = popen("ls -l", "r")
- pclose(p) = close pipe + wait for child + return exit code

pclose() → close file ptr, waitpid(), return exit code
fclose() → close stream, zombie child.

close(fd) | dup2(fd,0) dup2(fd,1) | 2
read(fd, buf, n)
write(fd, buf, n)
→ only close the file stream

A&&B&&C | dup2(pipefd[0])
A||B||C

0 = stdin
1 = stdout
2 = stderror

FILE* | STD FILENO
↳ descriptor

- → fclose(f) → flush buf, close FILE*
- fscanf(f, ...)
- fprintf(f, ...)
- fgets(buf, size, f)
- fgetc(f)

POPEN
→ create pipe
→ fork
→ child
→ dup2
→ exec
→ parent
→ fdopen

PCLOSE
→ fd close
→ waitpid
→ return status

read → from disk handlers
write → to data storage

FILE *p = popen("sort", "w")
→ pclose(p)
→ fgets(buf, size, p), fscanf(...)
→ pclose(p)
→ fprintf(p, ...); → to child process

stdout → terminal => DUP2
=> stdout → pipe

INPUT/OUTPUT/redirection

cmd > file
cmd >> file

cmd > file 2>&1
same place as stdout

tee ↳ stdout => screen, stdout => file

< file => stdin comes from file (cmd reads input from file)
> file => stdout to file (create if missing) (overwrite if ∃)
>> file => ||-|| (append if ∃)
cmd1 | cmd2 => stdout of cmd1 becomes stdin of cmd2

cmd 2>&1 >/dev/null | program

Send string to command using popen: (grep)
FILE* f = popen("grep", "w")
fprintf(f, "string");
pclose(f);

pipe(fd); fd[0]=R, fd[1]=W;
reader: close(fd[1]);
writer: close(fd[0]) dup2(fd[0], STDIN_FILENO) dup2(fd[1], STDOUT_FILENO) | dup2(old, new)

Redirect stdin
int fd = open("...", O_RDONLY);
dup2(fd, STDIN_FILENO);
close(fd);
execlp("cat", "cat", NULL)

if(fork()==0){ close(p[1]); dup2(p[0],0); close(p[0]); //read stdin...
else{ //parent=writer close(p[0]); dup2(p[1],1); close(p[1]); //printf... writes to the pipe; fflush(stdout); wait(0);}

DUP

printf to console
parent: stdout → pipe write end; printf writes to pipe
child: stdin → pipe read end
- fgets reads from pipe; printf to console

CREATE empty file

1) touch file; 2) > file; 3) echo -n > file; 4) nano file

fdopen: FILE* stream = fdopen(pipefd[0], "r").

EXEC = replace current process image with a new program (same PID)
↳ success => never returns;
↳ fail => returns -1 => next line

(to find) → argument
→ full path required

EXECL → args as a list => execl("/bin/ls", "ls", "-l", NULL);
EXECLP → search args in path => execlp("grep", "grep", "main", "file.c", NULL)
↳ when fixed nr. of args known => execlp("ls", "ls", NULL) => searches path
EXECV → args as a vector => execv("/bin/ls", argv) → when the nr. of args is dyn. (or known only at runtime)
EXECVP → search args in path => execvp("ls", argv*)
execvp(argv[1], argv+1) | execvp(argv[1], argv+1);

./a ls -l /etc/passwd

Zombie

1) parent calls wait() / waitpid() after child exits
2) signal(SIGCHILD, SIG_IGN) before creating children

ZOMBIE = process that finished execution but still appears in the pr. table
ORPHAN = still executes after parent is dead.
→ parent cannot create second child until C1 finishes
wait(0) => parent has children

RUN → WAIT (CPU used somewhere else while waiting for I/O op)
WAIT → RUN (when event becomes available) → program loaded into memory, waiting for CPU (OS scheduler decides)
READY → RUN (when it received the CPU)
RUN → READY (when the pr. is preempted, CPU given to sb else)

causes: 1) RR clock (q. exp.) 2) higher-priority process becomes ready

PROCESSES

= programs in execution (running)

- fork() => creates child process → > 0 => parent; = 0 => child; -1 => error
- ~~PID gets assigned to parent;~~ (child) => returns the child's PID to the parent
- child's PPID = parent's PID | wait(0) → keep exit code!
- wait(0) = wait(NULL) => parent waits for child to finish => removes zombie eventually; => => returns child's PID, -1 if no children.
- exit(status) => child must do that => terminate current process; status = 0 => success => back to parent

THREADS

= independent execution units within the same process: each thread has its own stack but share same mem. space, fd, resources.

- contain one or more threads (sep. memory) ≠
- have their own memory space (reduce mem. corr.)
- can be distributed to multiple machines
- better security (they don't share memory)
- a crash in one process won't affect the other

pthread_t thread;
void* f(void *arg); → writes the returned value here.
pthread_create(&tid, NULL, f, arg);
pthread_join(tid, NULL); → wait for thread to finish
pthread_exit(NULL) / return NULL
pthread_create(&t, NULL, f, &i) → addr. of the same var!
→ run-block-ready-run → waits for target thread to finish → → unblock when target thread finished...

recheck after pthread_cond_wait bc < signal does not guarantee condition is true, another t. may have ch.

- quicker to create than processes, more lightweight
- quicker to switch between threads (fewer sys. res.)
- share data easily (share same memory space) fd, res.)
- no security between threads • limited to one machine
- if one thread blocks, all threads in that process will block.
- concurrently perform tasks within a single program

pthread_barrier_wait(&bar) int.w.1 => release waiting thread (cond sched)

LOCK

= a variable < available (unlocked) / aquired (locked) } (1) mutual excl. (2) fairness (3) performance

POSIX

MUTEXES

→ threads claim exclusive control of resources that they require → one thr. at a time

PTHREAD_MUTEX_INITIALIZER
pthread_mutex_t m;
pthread_mutex_t m;
pthread_mutex_init(&m, NULL)
pthread_mutex_lock(&m);
pthread_mutex_unlock(&m);
pthread_mutex_destroy(&m);

sync. obj. change
exclusive

increase sem => release perm ≠ lock (enter protected section)

RACE CONDITION
(same var. protected by ≠ mutexes)
many W, few R => use rwlock

RWLOCKS

→ many reads one write
→ writer lock requires same thread to unlock it; sem: one lock ++, a ≠ lock --

pthread_rwlock_t rw;
pthread_rwlock_rdlock(&rw)
pthread_rwlock_wrlock(&rw)
(more work) (one thread enters only)

more readers incrementer.

BARRIERS

→ all threads wait until all arrive; → then continue together
_t, _init, _wait, _destroy
- blocked → (waited barrier)
- ready → last arrives
- running state

SEMAPHORES

allow m threads → simultan.

sem_t s;
sem_init(&s, 0, val); 0 = same process; 1 = separate processes
(P) sem_wait(&s); → cnt decreases, allows to enter; <0 => blo
(V) → sem_post(&s); → cnt increases, thread has left
PIPE sem_destroy(&s);
→ how many threads can enter
(sem = an int. whose value is not allowed to fall below 0; if it does => block/wait)
↳ Binary semaphore: sem_init(&s, 0, 1) ⇔ mutex
→ control access to a shared memory resource
Ordering semaphore: sem_init(..., 0) (one reader at a time)

OS SIGNALS

→ does not signal (kill signals)

signal(sig, handler) → installs a signal routine/handler
sig: SIGTERM (graceful), ...
SIGKILL (forced), SIGINT (ctrl+C),
SIGCHILD (child terminated),
kill(pid, sig) → sends signal sig to process pid;

CONDITIONAL VARIABLES

→ thread waits for its turn | - alternate - order - mutex

pthread_cond_t c; = PTHREAD_COND_IN
pthread_cond_init(&c, NULL);
pthread_cond_wait(&c, &m) | WHILE NOT IF
↳ unlocks the mutex, puts the thread to sleep, when signaled it wakes up the thread and locks the mutex before returning → run
pthread_cond_signal(&cond)
↳ wakes one waiting thread; ALL
pthread_cond_broadcast(&cond)

PIPES

pipe(int p[2]); p[0] = read end; p[1] = write end
read(p[0], buf, n)
write(p[1], buf, n)
{ dup2(pipe[1], 1) → copy old arg into new arg. | dup → to the first free | barrier_init(&b1, NULL, N) → → blocks when N ≠ arrive → newfd = dup(fd) ↓ automatic descriptor

=> one byte stream
int a2b = open
- empty pipe + ∃W => block R until ∃ data
- empty pipe + ∄W => read() returns 0 (EOF)
- full pipe + ∃R => block W.
- full pipe + ∄R => SIGPIPE / EPIPE

CLOSE PIPES: close(p[0]) (p[1])

FIFOS

= named pipes => one byte stream
O_RDONLY → open blocks until FIFO is opened for writing
int a2b = open("x", O_WRONLY) → open blocks until FIFO is opened for reading
mkfifo("fifoA", 0666) ↳ returns the read data or 0 if ∄ pr. to write (nobody write)
read(a2b, buf, sizeof(buf)) → waits until there is no pr. open for W or until there is data
write(a2b, buf, nbytes) → ∞ wait, until someone opens the read fifo.
close(a2b) →

- when unrelated processes need to communicate
- don't require common ancestor process
- FIFOs persist after processes terminate, while pipes ∃ only when their processes ∃! => pipe dies

a fifo can open as many fifos as file desc. limit

ulimit -n | for non-blocking

mkfifo("fifoA", 0600), int fd12 = open("fifoA", O_WRONLY) | fork

turnaround => when job process dies

RR → worst for turnaround

DEADLOCKS

→ process thr. enters an ∞ waiting state because needs a res. held by another.
EX: T1 holds L1 and waits for L2
T2 holds L2 and waits for L1

1) mutual exclusion
2) hold-and-wait
3) no preemption
4) circular wait

2) threads hold resources while waiting for other resources → sol: aquire all locks at once (prevention lock; unlock)
3) resources cannot forcibly be removed from threads that are holding them → sol: tryLOCK() + release (ex: lock(L1), if tryLOCK(L2) fails → unlock(L1) + retry.)
4) ∃ a circular chain of threads s.t. each thread holds one or more res. that are requested by the next thread in the chain → sol: lock in order

UNIX file system layout

BLOCK0 = boot block
BLOCK1 = super block
BLOCK2 ... n = inode blocks } map
BLOCK n+1 ... = data blocks } map

(5) next fit: where left off (4) worst fit: largest block (3) best fit: smallest block

INODE

0 | 1 | ... | 9 | 10 | 11 | 12 |
direct data block pointers | single indir. | double indir. | triple indir.

$\left(\frac{B}{A}\right)^m$ data blocks
N^m ($N^2 + N^3$ ...) addresses

Inode = metadata:
→ length
→ permissions
→ location (index node)

REPLACEMENT POLICIES

how to handle malloc) → does not search whole list

1) First Fit: choose first block large enough (+) fast (−) no attempt to ctrl. fragm.
2) Best Fit: smallest block large enough (+) economic in terms of large mem. areas (−) slower (−) creates small areas of free memory (fragmentation)
3) Worst Fit: largest free block (+) leaves behind larger chunks (−) slow
4) Buddy: allocate a chunk of the smallest pw of 2, greater than the required size; keep lists of free chunks by pw of 2, allocate the largest available chunk, distribute the remaining memory in pw of 2 chunks (+) speed

Producer: sem_wait(empty), lock(m), insert, unlock(m), post(full)
Consumer: sem_wait(full), lock(m), remove, unlock(m), post(empty)

PAGE REPLACEMENT POLICIES

1) FIFO: evict oldest loaded page
2) NRU (not recently used): class 0: (0,0); ref, modif. → choose victim from the lowest-numbered ≠ 0 class
3) LRU (last recently used) → evict pages not used for the longest time

CPU SCHEDULING POLICIES

1) FCFS / FIFO → (+) simple; (−) convoy, (−) need to know job durations
2) SJF (sh. job first) → non-preemptive; does not interrupt a running job
↳ smallest TOTAL runtime, → smallest REMAINING runtime
3) STCF (sh. time to completion first) → preemptive; can interrupt a run.
4) RR clock → preemptive → each process gets a quantum of time (simultan)
5) Priority → highest priority first → (−) starvation
6) MLFQ (multi-level feedback queue) (+) good CPU utilis. (−) q. too small => many context switches
→ pr. placed in > priority queues → new pr. placed in highest p.q. → lower priority (+) no starvation (when t. quant. used) (−) q. too large => behaves like FCFS
• ∃ I/O op. or interaction => stay near top; • CPU-bound pr => move down (prior)
7) Lottery → percent of tickets = share of the system res. → know total tickets → pick random → probabilistically
8) Stride → deterministic lottery (each job has a stride, i.p. to the nr. of tickets) ll → smaller is better
9) EDD → earliest due date (earliest deadline) BADC (4<7<8<13)
10) EDF → earliest deadline first (+) real-time scheduling (Deadline) (−) missing deadline = fail | CFS → fair share sch. → weighted RR
(11) CFS = fair-share scheduling (real scheduler)

LOCALITY PRINCIPLE (optimisation)

- SPATIAL => nearby pages loaded; prefetch
- TEMPORAL => recently-used pages loaded;

The pages neighbouring the page that has just been loaded are more likely to be needed so we load them preemptively
BEHAND-PAGING load(=) pages are needed.

LOADING POLICIES

1) all pages in the beg
(+) fast access at runtime
(−) slow startup (−) memory
(−) loading pages not needed
2) each page when needed
(−) slow access at runtime
(+) fast startup (+) loading what needed.